

Full Stack Web Development





CHAPTER-1

Java Networking & RMI





Java Networking

Java Networking is a concept of connecting two or more computing devices together so that we can share resources.

Java socket programming provides facility to share data between different computing devices.

Advantage of Java Networking

Sharing resources

Centralize software management

The java.net package supports two protocols,

TCP: Transmission Control Protocol provides reliable communication between the sender and receiver. TCP is used along with the Internet Protocol referred as TCP/IP.

UDP: User Datagram Protocol provides a connection-less protocol service by allowing packet of data to be transferred along two or more nodes.

Java Networking Terminology

The widely used Java networking terminologies are given below:

1. IP Address
2. Protocol
3. Port Number
4. MAC Address
5. Connection-oriented and connection-less protocol
6. Socket

1) IP Address

IP address is a unique number assigned to a node of a network e.g. 192.168.0.1 . It is composed of octets that range from 0 to 255. It is a logical address that can be changed.



Contin.

2) Protocol

A protocol is a set of rules basically that is followed for communication. For example:

- TCP
- FTP
- Telnet
- SMTP
- POP etc.

3) Port Number

The port number is used to uniquely identify different applications. It acts as a communication endpoint between applications.

The port number is associated with the IP address for communication between two applications.



conti

4) MAC Address

MAC (Media Access Control) address is a unique identifier of NIC (Network Interface Controller). A network node can have multiple NIC but each with unique MAC address.

For example, an ethernet card may have a **MAC** address of 00:0d:83::b1:c0:8e.

5) Connection-oriented and connection-less protocol

In connection-oriented protocol, acknowledgement is sent by the receiver. So it is reliable but slow. The example of connection-oriented protocol is TCP.

But, in connection-less protocol, acknowledgement is not sent by the receiver. So it is not reliable but fast. The example of connection-less protocol is UDP.

6) Socket

A socket is an endpoint between two way communications.

Visit next page for Java socket programming.



java.net package

The java.net package can be divided into two sections:

A Low-Level API: It deals with the abstractions of addresses i.e. networking identifiers, Sockets i.e. bidirectional data communication mechanism and Interfaces i.e. network interfaces.

A High Level API: It deals with the abstraction of URIs i.e. Universal Resource Identifier, URLs i.e. Universal Resource Locator, and Connections i.e. connections to the resource pointed by URLs.

The java.net package provides many classes to deal with networking applications in Java. A list of these classes is given below:

- | | | |
|----------------|--------------------------|----------------------|
| Authenticator | • HttpURLConnection | • DatagramSocket |
| CacheRequest | • HttpCookie | • DatagramSocketImpl |
| CacheResponse | • NetPermission | • InterfaceAddress |
| ContentHandler | • NetworkInterface | • JarURLConnection |
| CookieHandler | • PasswordAuthentication | • MulticastSocket |
| CookieManager | • Proxy | • InetSocketAddress |
| DatagramPacket | • ProxySelector | • InetAddress |
| | | • Inet4Address |



conti

- ResponseCache
- SecureCacheResponse
- ServerSocket
- Socket
- SocketAddress
- SocketImpl
- SocketPermission
- StandardSocketOptions
- URLClassLoader
- URLConnection
- URLDecoder
- URLEncoder
- URLStreamHandler

List of interfaces available in java.net package:

- ContentHandlerFactory
- CookiePolicy
- CookieStore
- DatagramSocketImplFactory
- FileNameMap
- SocketOption<T>
- SocketOptions
- SocketImplFactory
- URLStreamHandlerFactory
- ProtocolFamily



Java Socket Programming

Java Socket programming is used for communication between the applications running on different JRE.

Java Socket programming can be connection-oriented or connection-less.

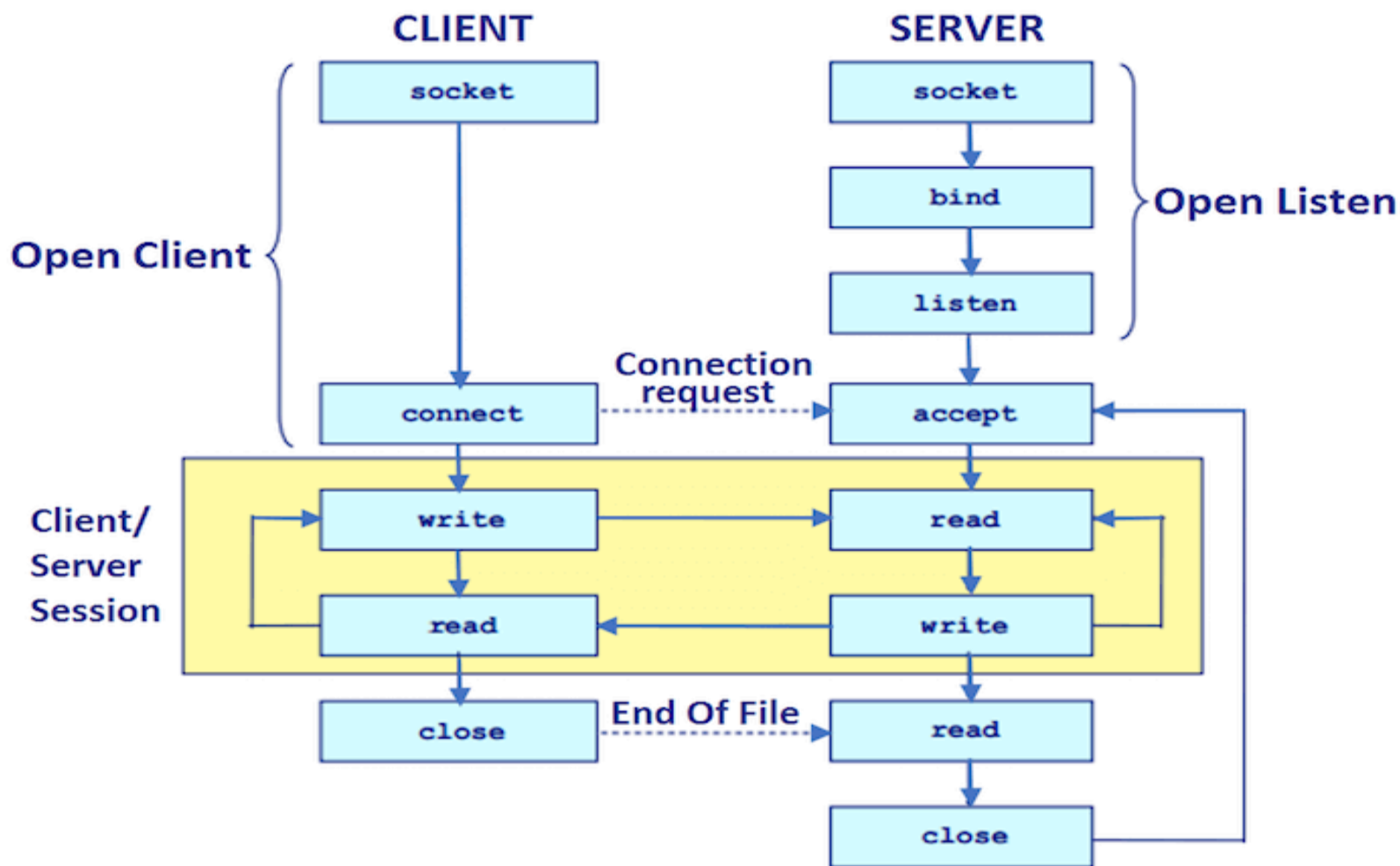
Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket and DatagramPacket classes are used for connection-less socket programming.

The client in socket programming must know two information:

IP Address of Server, and

Port number.

Here, we are going to make one-way client and server communication. In this application, client sends a message to the server, server reads the message and prints it. Here, two classes are being used: Socket and ServerSocket. The Socket class is used to communicate client and server. Through this class, we can read and write message. The ServerSocket class is used at server-side. The accept() method of ServerSocket class blocks the console until the client is connected. After the successful connection of client, it returns the instance of Socket at server-side.



Socket class

A socket is simply an endpoint for communications between the machines. The Socket class can be used to create a socket.

Important methods

Method	Description
1) public InputStream getInputStream()	returns the InputStream attached with this socket.
2) public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
3) public synchronized void close()	closes this socket



ServerSocket class

The ServerSocket class can be used to create a server socket. This object is used to establish communication with the clients.

Important methods

Method	Description
1) public Socket accept()	returns the socket and establish a connection between server and client.
2) public synchronized void close()	closes the server socket.



Example of Java Socket Programming

Creating Server:

To create the server application, we need to create the instance of `ServerSocket` class. Here, we are using 6666 port number for the communication between the client and server. You may also choose any other port number. The `accept()` method waits for the client. If clients connects with the given port number, it returns an instance of `Socket`.

```
ServerSocket ss=new ServerSocket(6666);
```

```
Socket s=ss.accept();//establishes connection and waits for the client
```

Creating Client:

To create the client application, we need to create the instance of `Socket` class. Here, we need to pass the IP address or hostname of the Server and a port number. Here, we are using "localhost" because our server is running on same system.



Example File: MyServer.

Let's see a simple of Java socket programming where client sends a text and server receives and prints it.

```
import java.io.*;
import java.net.*;
public class MyServer {
    public static void main(String[] args){
    try{
        ServerSocket ss=new ServerSocket(6666);
        Socket s=ss.accept();//establishes connection
        DataInputStream dis=new DataInputStream(s.getInputStream());
        String str=(String)dis.readUTF();
        System.out.println("message= "+str);
        ss.close();
    }catch(Exception e){System.out.println(e);}
}
```



File: MyClient.java

```
import java.io.*;
import java.net.*;
public class MyClient {
public static void main(String[] args) {
try{
    Socket s=new Socket("localhost",6666);
    DataOutputStream dout=new DataOutputStream(s.getOutputStream());
    dout.writeUTF("Hello Server");
    dout.flush();
    dout.close();
    s.close();
catch(Exception e){System.out.println(e);}
}
}
```

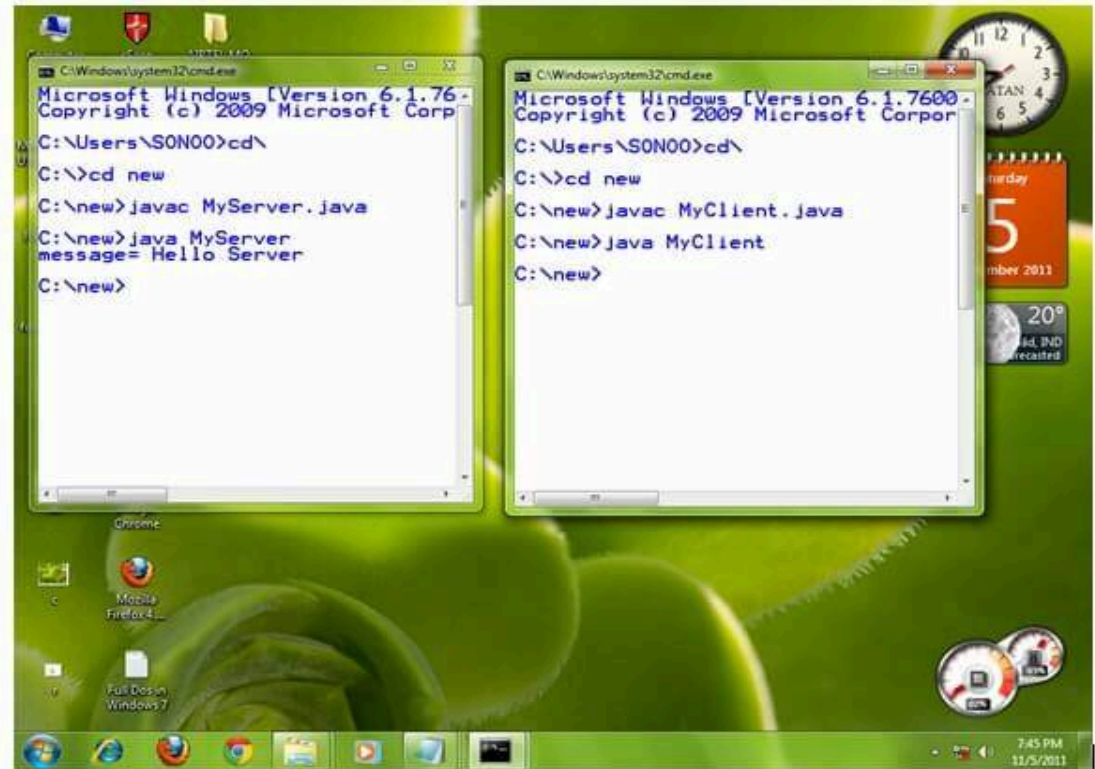




How to Execute

To execute this program open two command prompts and execute each program at each command prompt as displayed in the below figure.

After running the client application, a message will be displayed on the server console.



Example of Java Socket Programming (Read-Write both side)

In this example, client will write first to the server then server will receive and print the text. Then server will write to the client and client will receive and print the text. The step goes on.

File: MyServer.java

```
import java.net.*;
import java.io.*;
class MyServer{
public static void main(String args[])throws Exception{
    ServerSocket ss=new ServerSocket(3333);
    Socket s=ss.accept();
    DataInputStream din=new DataInputStream(s.getInputStream());
    DataOutputStream dout=new DataOutputStream(s.getOutputStream());
    BufferedReader br=new BufferedReader(new InputStreamReader(Syste
```

conti

```
String str="",str2="";  
while(!str.equals("stop")){  
    str=din.readUTF();  
    System.out.println("client says:  
        "+str);  
    str2=br.readLine();  
    dout.writeUTF(str2);  
    dout.flush();  
}  
din.close();  
s.close();  
ss.close();  
}}
```



conti

File: MyClient.java

```
import java.net.*;
```

```
import java.io.*;
```

```
class MyClient{
```

```
public static void main(String args[])throws Exception{
```

```
Socket s=new Socket("localhost",3333);
```

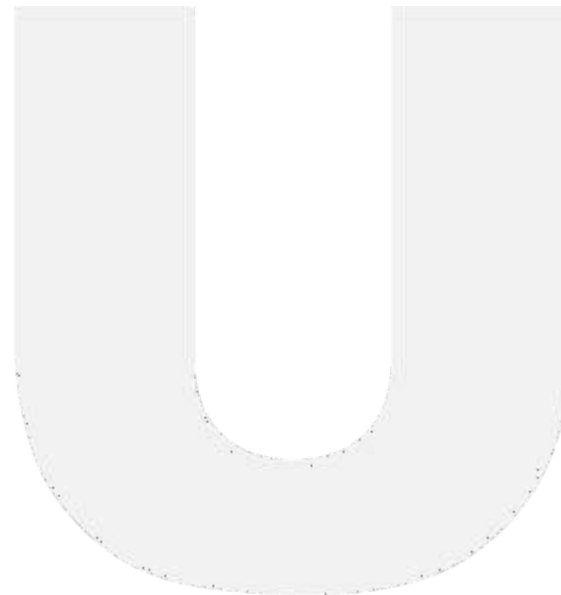
```
DataInputStream din=new DataInputStream(s.getInputStream());
```

```
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
```

```
BufferedReader br=new BufferedReader(new InputStreamReader(System.  
in));
```


conti

```
String str="",str2="";  
while(!str.equals("stop")){  
    str=br.readLine();  
    dout.writeUTF(str);  
    dout.flush();  
    str2=din.readUTF();  
    System.out.println("Server says:  
        "+str2);  
}  
  
dout.close();  
s.close();  
}}
```



Java URL

The **Java URL** class represents an URL. URL is an acronym for Uniform Resource Locator. It points to a resource on the World Wide Web.

A URL contains many information:

Protocol: In this case, http is the protocol.

Server name or IP Address: In this case, www.javatpoint.com is the server name.

Port Number: It is an optional attribute. If we write http://www.javatpoint.com:80/sonoojaiswal/ , 80 is the port number. If port number is not mentioned in the URL, it returns -1.

File Name or directory name: In this case, index.jsp is the file name.



Constructors of Java URL class

URL(String spec)

Creates an instance of a URL from the String representation.

URL(String protocol, String host, int port, String file)

Creates an instance of a URL from the given protocol, host, port number, and file.

URL(String protocol, String host, int port, String file, URLStreamHandler handler)

Creates an instance of a URL from the given protocol, host, port number, file, and handler.

URL(String protocol, String host, String file)

Creates an instance of a URL from the given protocol name, host name, and file name.

URL(URL context, String spec)

Creates an instance of a URL by parsing the given spec within a specified context.

URL(URL context, String spec, URLStreamHandler handler)

Creates an instance of a URL by parsing the given spec with the specified



Commonly used methods of Java URL class

- The java.net.URL class provides many methods. The important methods of URL class are given below

Method	Description
public String getProtocol()	it returns the protocol of the URL.
public String getHost()	it returns the host name of the URL.
public String getPort()	it returns the Port Number of the URL.
public String getFile()	it returns the file name of the URL.
public String getAuthority()	it returns the authority of the URL.
public String toString()	it returns the string representation of the URL.
public String getQuery()	it returns the query string of the URL.



conti

`public String getDefaultPort()`

it returns the default port of the URL.

`public URLConnection
openConnection()`

it returns the instance of
URLConnection i.e. associated with
this URL.

`public boolean equals(Object obj)`

it compares the URL with the given
object.

`public Object getContent()`

it returns the content of the URL.

`public String getRef()`

it returns the anchor or reference of
the URL.

`public URI toURI()`

it returns a URI of the URL.

Example of Java URL class

```
//URLDemo.java
import java.net.*;
public class URLDemo{
public static void main(String[] args){
try{
URL url=new URL("http://www.javatpoint.com/java-tutorial");
System.out.println("Protocol: "+url.getProtocol());
System.out.println("Host Name: "+url.getHost());
System.out.println("Port Number: "+url.getPort());
System.out.println("File Name: "+url.getFile());
}catch(Exception e){System.out.println(e);}
}
}
```

Protocol: http

Host Name: www.javatpoint.
com

Port Number: -1

File Name: /java-tutorial



Example URL class in

```
//URLDemo.java
import java.net.*;
public class URLDemo{
public static void main(String[] args){
try{
URL url=new URL("https://www.google.com/
search?q=javatpoint&oq=javatpoint&sourceid=chrome&ie=UTF-8");
System.out.println("Protocol: "+url.getProtocol());
System.out.println("Host Name: "+url.getHost());
System.out.println("Port Number: "+url.getPort());
System.out.println("Default Port Number: "+url.getDefaultPort());
System.out.println("Query String: "+url.getQuery());
System.out.println("Path: "+url.getPath());
System.out.println("File: "+url.getFile());
}catch(Exception e){System.out.println(e);}
} }
```



o/p

o/p
Protocol: *https*
Host Name: *www.google.com* Port Number: *-1*
Default Port Number: *443*
Query String: *q=javatpoint&oq=javatpoint&sourceid=chrome&ie=UTF-8*
Path: */search*
File: */search?q=javatpoint&oq=javatpoint&sourceid=chrome&ie=UTF-8*



Java URLConnection Class

The **Java URLConnection** class represents a communication link between the URL and the application. It can be used to read and write data to the specified resource referred by the URL.

What is the URL?

URL is an abbreviation for Uniform Resource Locator. An URL is a form of string that helps to find a resource on the World Wide Web (WWW).

URL has two components:

The protocol required to access the resource.

The location of the resource.

Features of URLConnection class

1. URLConnection is an abstract class. The two subclasses HttpURLConnection and JarURLConnection makes the connection between the client Java program and URL resource on the internet.
2. With the help of URLConnection class, a user can read and write to and from any resource referenced by an URL object.
3. Once a connection is established and the Java program has an URLConnection object, we can use it to read or write or get further information like content length, etc.



Constructors

Constructor

1) protected `URLConnection(URL url)`

Description

It constructs a URL connection to the specified URL.

URLConnection Class Methods





URLConnection Class Methods

Method	Description
<code>void addRequestProperty(String key, String value)</code>	It adds a general request property specified by a key-value pair
<code>void connect()</code>	It opens a communications link to the resource referenced by this URL, if such a connection has not already been established.
<code>boolean getAllowUserInteraction()</code>	It returns the value of the allowUserInteraction field for the object.
<code>int getConnectionTimeout()</code>	It returns setting for connect timeout.
<code>Object getContent()</code>	It retrieves the contents of the URL connection



How to get the object of URLConnection Class

The `openConnection()` method of the `URL` class returns the object of `URLConnection` class.

Syntax:

public `URLConnection` `openConnection()` **throws** `IOException` {

Displaying Source Code of a Webpage by `URLConnection` Class

The `URLConnection` class provides many methods. We can display all the data of a webpage by using the `getInputStream()` method. It returns all the data of the specified URL in the stream that can be read and displayed.

Example of Java URLConnection Class

```
import java.io.*;
import java.net.*;
public class URLConnectionExample {
    public static void main(String[] args){
    try{
        URL url=new URL("http://www.javatpoint.com/java-tutorial");
        URLConnection urlcon=url.openConnection();
        InputStream stream=urlcon.getInputStream();
        int i;
        while((i=stream.read())!=-1){
            System.out.print((char)i);
        }
    }catch(Exception e){System.out.println(e);}
    }
}
```

Java HttpURLConnection class

The **Java HttpURLConnection** class is http specific URLConnection. It works for HTTP protocol only.

By the help of HttpURLConnection class, you can retrieve information of any HTTP URL such as header information, status code, response code etc.

The java.net.HttpURLConnection is subclass of URLConnection class.

HttpURLConnection Class Constructor

Constructor	Description
protected HttpURLConnection(URL u)	It constructs the instance of HttpURLConnection class.

Java HttpURLConnection Methods

Method	Description
<code>void disconnect()</code>	It shows that other requests from the server are unlikely in the near future.
<code>InputStream getErrorStream()</code>	It returns the error stream if the connection failed but the server sent useful data.
<code>Static boolean getFollowRedirects()</code>	It returns a boolean value to check whether or not HTTP redirects should be automatically followed.
<code>String getHeaderField(int n)</code>	It returns the value of nth header file.
<code>long getHeaderFieldDate(String name, long Default)</code>	It returns the value of the named field parsed as a date.

How to get the object of HttpURLConnection class

The `openConnection()` method of `URL` class returns the object of `URLConnection` class.

Syntax:

public `URLConnection` `openConnection()` **throws** `IOException`{}

You can typecast it to `HttpURLConnection` type as given below.

```
URL url=new URL("http://www.javatpoint.com/java-tutorial");
```

```
HttpURLConnection huc=(HttpURLConnection)url.openConnection();
```

Java HttpURLConnection Example

```
import java.io.*;
import java.net.*;
public class HttpURLConnectionDemo{
public static void main(String[] args){
try{
URL url=new URL("http://www.javatpoint.com/java-tutorial");
HttpURLConnection huc=(HttpURLConnection)url.openConnection();
for(int i=1;i<=8;i++){
System.out.println(huc.getHeaderFieldKey(i)+" = "+huc.getHeaderField(i));
}
huc.disconnect();
}catch(Exception e){System.out.println(e);}
}
}
```


o/p

*Date = Thu, 22 Jul 2021 18:08:17
GMT*

Server = Apache

Location = java-tutorial

Cache-Control = max-age=2592000

*Expires = Sat, 21 Aug 2021 18:08:17
GMT*

Content-Length = 248

*Keep-Alive =timeout=5,
max=1500*

Connection = Keep-Alive



o/p

```
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\WIN 8.1>cd Desktop

C:\Users\WIN 8.1\Desktop>javac HttpURLConnectionDemo.java

C:\Users\WIN 8.1\Desktop>java HttpURLConnectionDemo
Date = Thu, 22 Jul 2021 18:08:17 GMT
Server = Apache
Location = https://www.javatpoint.com/java-tutorial
Cache-Control = max-age=2592000
Expires = Sat, 21 Aug 2021 18:08:17 GMT
Content-Length = 248
Keep-Alive = timeout=5, max=1500
Connection = Keep-Alive

C:\Users\WIN 8.1\Desktop>
```

Java InetAddress class

Java InetAddress class represents an IP address. The `java.net.InetAddress` class provides methods to get the IP of any host name *for example* `www.javatpoint.com`, `www.google.com`, `www.facebook.com`, etc.

An IP address is represented by 32-bit or 128-bit unsigned number. An instance of `InetAddress` represents the IP address with its corresponding host name. There are two types of addresses: Unicast and Multicast. The Unicast is an identifier for a single interface whereas Multicast is an identifier for a set of interfaces.

Moreover, `InetAddress` has a cache mechanism to store successful and unsuccessful host name resolutions.

IP Address

- An IP address helps to identify a specific resource on the network using a numerical representation.
- Most networks combine IP with TCP (Transmission Control Protocol). It builds a virtual bridge among the destination and the source.

PU



There are two versions of IP addresses

1. IPv4

IPv4 is the primary Internet protocol. It is the first version of IP deployed for production in the ARPANET in 1983. It is a widely used IP version to differentiate devices on network using an addressing scheme. A 32-bit addressing scheme is used to store 2^{32} addresses that is more than 4 million addresses.

Features of IPv4:

- It is a connectionless protocol.
- It utilizes less memory and the addresses can be remembered easily with the class based addressing scheme.
- It also offers video conferencing and libraries.

conti

2. IPv6

IPv6 is the latest version of Internet protocol. It aims at fulfilling the need of more internet addresses. It provides solutions for the problems present in IPv4. It provides 128-bit address space that can be used to form a network of 340 undecillion unique IP addresses. IPv6 is also identified with a name IPng (Internet Protocol next generation).

Features of IPv6:

- It has a stateful and stateless both configurations.
- It provides support for quality of service (QoS).
- It has a hierarchical addressing and routing infrastructure.

conti

TCP/IP Protocol

- TCP/IP is a communication protocol model used connect devices over a network via internet.
- TCP/IP helps in the process of addressing, transmitting, routing and receiving the data packets over the internet.
- The two main protocols used in this communication model are:
 - TCP i.e. Transmission Control Protocol. TCP provides the way to create a communication channel across the network. It also helps in transmission of packets at sender end as well as receiver end.
 - IP i.e. Internet Protocol. IP provides the address to the nodes connected on the internet. It uses a gateway computer to check whether the IP address is correct and the message is forwarded correctly or not.

Java InetAddress Class Methods

The InetAddress class has no visible constructors. The class has some static methods.

Method	Description
<code>public static InetAddress getByName(String host) throws UnknownHostException</code>	It returns the instance of InetAddress containing LocalHost IP and name.
<code>public static InetAddress getLocalHost() throws UnknownHostException</code>	It returns the instance of InetAddress containing local host name and address.
<code>public String getHostName()</code>	It returns the host name of the IP address.
<code>public String.getHostAddress()</code>	It returns the IP address in string format.

Example of Java InetAddress Class

Let's see a simple example of InetAddress class to get ip address of www.javatpoint.com website.

InetDemo.java

```
import java.io.*;
import java.net.*;
public class InetDemo{
public static void main(String[] args){
try{
InetAddress ip=InetAddress.getByName("www.javatpoint.com");
System.out.println("Host Name: "+ip.getHostName());
System.out.println("IP Address: "+ip.getHostAddress());
}catch(Exception e){System.out.println(e);}
}
}
```

```
o/p
Host Name: www.javatpoint.
com
IP Address: 172.67.196.82
```

Program to demonstrate methods of InetAddress class

InetDemo2.java

```
import java.net.Inet4Address;
import java.util.Arrays;
import java.net.InetAddress;
public class InetDemo2 {
public static void main(String[] arg) throws Exception {
    InetAddress ip = Inet4Address.getByName("www.javatpoint.com");
    InetAddress ip1[] = InetAddress.getAllByName("www.javatpoint.com");
    byte addr[]={72, 3, 2, 12};
    System.out.println("ip : "+ip);
    System.out.print("\nip1 : "+ip1);
    InetAddress ip2 = InetAddress.getByAddress(addr);
    System.out.print("\nip2 : "+ip2);
```



conti

```
System.out.print("\nAddress : " + Arrays.toString(ip.getAddress()));  
System.out.print("\nHost Address : " + ip.getHostAddress());  
System.out.print("\nisAnyLocalAddress : " + ip.isAnyLocalAddress());  
System.out.print("\nisLinkLocalAddress : " + ip.isLinkLocalAddress());  
System.out.print("\nisLoopbackAddress : " + ip.isLoopbackAddress());  
System.out.print("\nisMCGlobal : " + ip.isMCGlobal());  
System.out.print("\nisMCLinkLocal : " + ip.isMCLinkLocal());  
System.out.print("\nisMCNodeLocal : " + ip.isMCNodeLocal());  
System.out.print("\nisMCOrgLocal : " + ip.isMCOrgLocal());  
System.out.print("\nisMCSiteLocal : " + ip.isMCSiteLocal());  
System.out.print("\nisMulticastAddress : " + ip.isMulticastAddress());  
System.out.print("\nisSiteLocalAddress : " + ip.isSiteLocalAddress());  
System.out.print("\nhashCode : " + ip.hashCode());  
System.out.print("\n Is ip1 == ip2 : " + ip.equals(ip2));  
}}
```



o/p

```
Command Prompt

C:\Users\WIN 8.1>cd Desktop

C:\Users\WIN 8.1\Desktop>javac InetDemo2.java

C:\Users\WIN 8.1\Desktop>java InetDemo2
ip : www.javatpoint.com/172.67.196.82

ip1 : [Ljava.net.InetAddress;@7852e922
ip2 : /72.3.2.12
Address : [-84, 67, -60, 82]
Host Address : 172.67.196.82
isAnyLocalAddress : false
isLinkLocalAddress : false
isLoopbackAddress : false
isMCGlobal : false
isMCLinkLocal : false
isMCNodeLocal : false
isMCOrgLocal : false
isMCSiteLocal : false
isMulticastAddress : false
isSiteLocalAddress : false
hashCode : -1404844974
Is ip1 == ip2 : false
C:\Users\WIN 8.1\Desktop>
```



Java DatagramSocket and DatagramPacket

Java DatagramSocket and DatagramPacket classes are used for connection-less socket programming using the UDP instead of TCP.

Datagram

Datagrams are collection of information sent from one device to another device via the established network. When the datagram is sent to the targeted device, there is no assurance that it will reach to the target device safely and completely. It may get damaged or lost in between. Likewise, the receiving device also never know if the datagram received is damaged or not. The UDP protocol is used to implement the datagrams in Java.

conti

Java DatagramSocket class

Java DatagramSocket class represents a connection-less socket for sending and receiving datagram packets. It is a mechanism used for transmitting datagram packets over network.

A datagram is basically an information but there is no guarantee of its content, arrival or arrival time.

Commonly used Constructors of DatagramSocket class

- **DatagramSocket()** throws **SocketEeption**: it creates a datagram socket and binds it with the available Port Number on the localhost machine.
- **DatagramSocket(int port)** throws **SocketEeption**: it creates a datagram socket and binds it with the given Port Number.
- **DatagramSocket(int port, InetAddress address)** throws **SocketEeption**: it creates a datagram socket and binds it with the specified port number and host address

Java DatagramSocket Class

Method	Description
<code>void bind(SocketAddress addr)</code>	It binds the DatagramSocket to a specific address and port.
<code>void close()</code>	It closes the datagram socket.
<code>void connect(InetAddress address, int port)</code>	It connects the socket to a remote address for the socket.
<code>void disconnect()</code>	It disconnects the socket.
<code>boolean getBroadcast()</code>	It tests if SO_BROADCAST is enabled.
<code>DatagramChannel getChannel()</code>	It returns the unique DatagramChannel object associated with the datagram socket.
<code>InetAddress getInetAddress()</code>	It returns the address to where the socket is connected.

Java DatagramPacket Class

Java DatagramPacket is a message that can be sent or received. It is a data container. If you send multiple packet, it may arrive in any order. Additionally, packet delivery is not guaranteed.

Commonly used Constructors of DatagramPacket class

- **DatagramPacket(byte[] barr, int length):** it creates a datagram packet. This constructor is used to receive the packets.
- **DatagramPacket(byte[] barr, int length, InetAddress address, int port):** it creates a datagram packet. This constructor is used to send the packets.

Java DatagramPacket Class Methods

Method	Description
1) InetAddress getAddress()	It returns the IP address of the machine to which the datagram is being sent or from which the datagram was received.
2) byte[] getData()	It returns the data buffer.
3) int getLength()	It returns the length of the data to be sent or the length of the data received.
4) int getOffset()	It returns the offset of the data to be sent or the offset of the data received.

Example of Sending DatagramPacket by DatagramSocket

```
//DSender.java
import java.net.*;
public class DSender{
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket();
        String str = "Welcome java";
        InetAddress ip = InetAddress.getByName("127.0.0.1");

        DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(), ip,
        3000);
        ds.send(dp);
        ds.close();
    }
}
```



o/p

```
Command Prompt

Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\WIN 8.1>cd Desktop
C:\Users\WIN 8.1\Desktop>javac DSender.java
C:\Users\WIN 8.1\Desktop>java DSender
C:\Users\WIN 8.1\Desktop>
```

Example of Receiving DatagramPacket by DatagramSocket

```
//DReceiver.java
import java.net.*;
public class DReceiver{
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket(3000);
        byte[] buf = new byte[1024];
        DatagramPacket dp = new DatagramPacket(buf, 1024);
        ds.receive(dp);
        String str = new String(dp.getData(), 0, dp.getLength());

        System.out.println(str);
        ds.close();
    }
}
```



o/p

```
Command Prompt
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\WIN 8.1>cd Desktop
C:\Users\WIN 8.1\Desktop>javac DReceiver.java
C:\Users\WIN 8.1\Desktop>java DReceiver
Welcome java
C:\Users\WIN 8.1\Desktop>
```


× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.

